Imam Abdulrahman Bin Faisal University
Deanship of Preparatory Year and Supporting Studies
Computer Department

Basics of Programming through Python
Repetition Structure



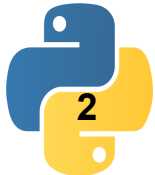
Introduction to Programming

COMP106

Term 1 | 2026-2027

Learning outcomes

- Write different types of loops, such as for, and while.
- Use loops to solve common problems, such as, finding sum, finding max, etc.
- Select the correct type of loop based on a given problem.



Looping

- Looping is the process in which we have a list of statements that executes repeatedly until it satisfies a condition.

- Python provides two kinds of loops :

  - while loop
  - for loop



while Loop

while loop

- A while loop is a control flow statement that allows code to be executed repeatedly while a given condition still holds true.
- Syntax:

```
while (condition):
    Statements
```

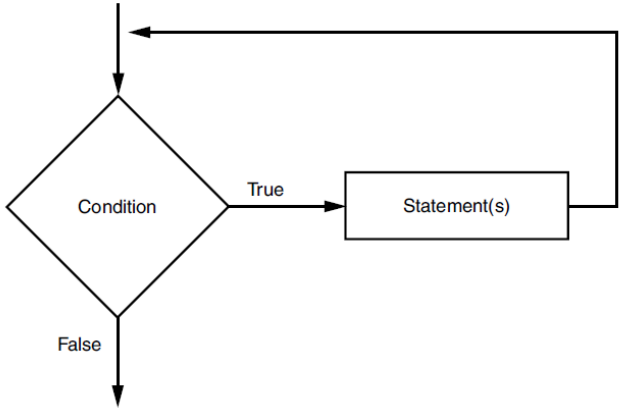
- Three components that you need to construct the while loop in Python:
  - The while: keyword.
  - A condition: that is evaluated to either True or False; And
  - A statements: block of code that you want to execute repeatedly



while loop

1. If the condition is false, exit the while statement and continue execution at the next statement.
2. If the condition is true, execute the body and then go back to step 1.

- → The while loop is known as a ***pretest*** loop, which means it tests its condition ***before*** performing an iteration.



General way to write a while loop

Initialize your counter (line 1)

- A variable is initialized outside of the loop

while condition: (line 2)

- While loop executes while the condition is True

Statements (line 3)

- Executes the code within the block of code

counter update (line 4)

- Counter is updated inside the loop

Instructions

- More instructions after the while loop(optional)

Example: 1

```
1   x = 1
2   while x < 5:
3         print(x)
4         x = x + 1
```

Output: 1

```
1
2
3
4
```



Updating variables

- Updating a variable by adding 1 is called an increment

  i = i + 1 or i += 1

- subtracting 1 is called a decrement

  i = i - 1 or i -= 1



Example (1)

```
x = 10
while (x != 0):
        print(x)
        x -= 1
```

Output :

```
10
9
8
7
6
5
4
3
2
1
```



Example (2)

```
sum1 = 0
count = 1
while (count < 10):
        sum1 = sum1 + count
        count += 1
print(count)
print(sum1)
```

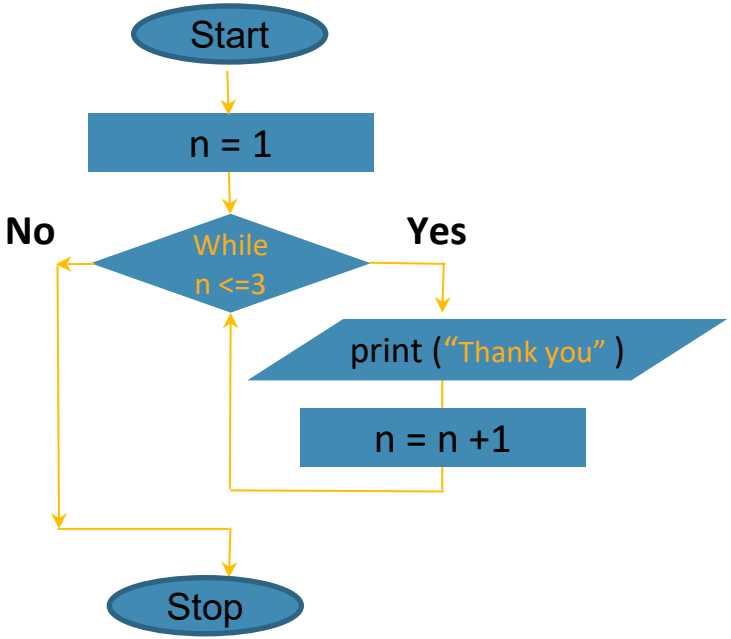
Output :

```
10
45
```



Example (3)

- Displaying "Thank you" 3 times



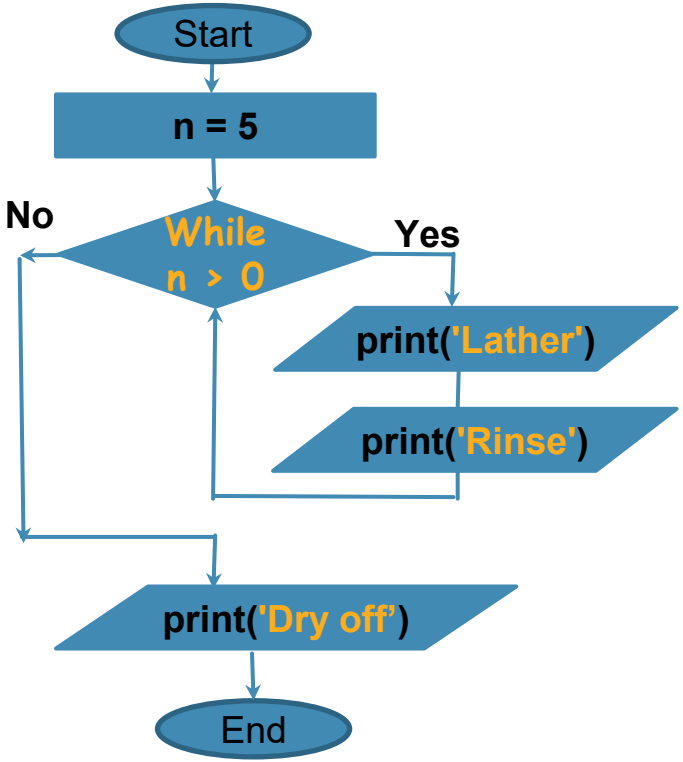
- Program:

```
# initialization
n = 1
# Condition of the while loop
while n <= 3
    print ("Thank you")
    # Increment the value of the variable "n by 1"
    n = n + 1
```

- Output:

```
Thank you
Thank you
Thank you
```

An Infinite Loop (NO counter update)



- Program:

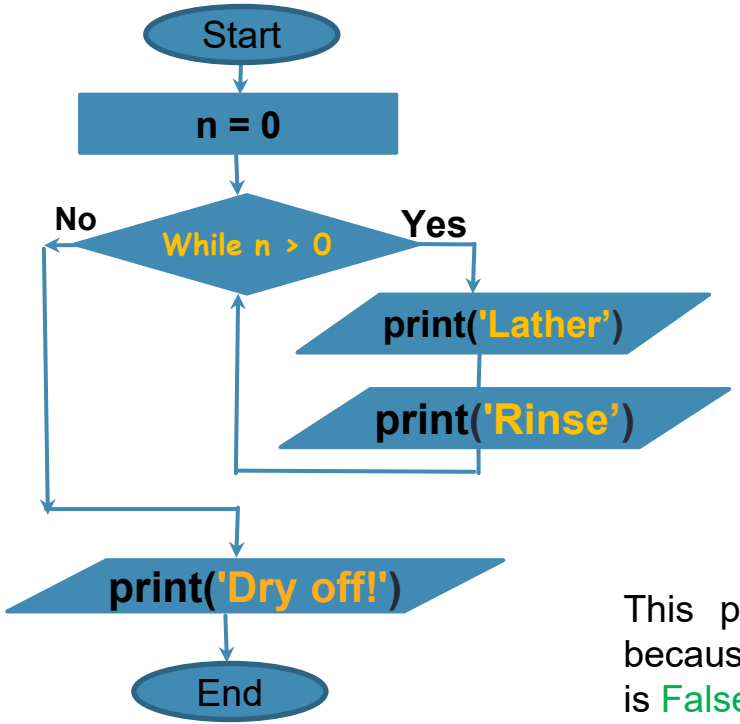
```
n = 5
while n > 0
    print ("Lather")
    print ("Rinse")
print ("Dry off")
```

- Output:

Lather
Rinse
Lather
Rinse
…

Will be infinitely displayed
and "Dry off" will never be displayed

Another Loop



- Program:

```
n = 0
while n > 0
    print ("Lather")
    print ("Rinse")
print ("Dry off")
```

- Output:

```
Dry off
```

This program will not display “Lather” and “Rinse” because the logical expression on the while statement is False from the first iteration

The break statement

- The break statement ends the current loop and jumps to the statement immediately following the loop.
- It is like a loop test that can happen anywhere in the body of the loop.

- Program:

```
x = 10
while x > 0:
    print ('Current x value :', x)
    x = x -1
    if x == 5:
        break
print("Goodbye!")
```

- Output:

```
Current x value : 10
Current x value : 9
Current x value : 8
Current x value : 7
Current x value : 6
Goodbye!
```



The continue statement

The continue statement ends the current iteration and jumps to the top of the loop and starts the next iteration.

- Example:

Write a Python program to print odd numbers from 1 to 10

- Program:

```
num = 0

while num < 10:
    num += 1

    if (num % 2) == 0:
        continue

    print(num)
```

- Output:

```
1
3
5
7
9
```



Difference Between break and continue

break	continue
It terminates the execution of remaining iteration of the loop.	It terminates only the current iteration of the loop.
'break' resumes the control of the program to the end of loop enclosing that 'break'.	'continue' resumes the control of the program to the next iteration of that loop enclosing 'continue'.
It causes early termination of loop.	It causes early execution of the next iteration.
'break' stops the continuation of loop.	'continue' do not stops the continuation of loop, it only stops the current iteration.

Output

- **Example1:**

```
var = 10
while var > 0:
   print ('Current variable value :', var)
   var = var -1
   if var == 5:
      break
print("Good bye!")
```

- **Example2:**

```
var = 10
while var > 0:
   var = var -1
   if var == 5:
      continue
   print ('Current variable value :', var)
print ("Good bye!")
```



Try

- Write a Python program that repeatedly asks the user to enter a response:
  - If the user enters "yes" or "no", the program should display the message: "Thank you for your response." and then stop.
  - If the user enters anything else, the program should display: "Invalid response. Please try again." and continue asking until a valid response is entered.



Answer

Simple Run1:

```
Please enter a valid response: y
Invalid response. Please try again.
Please enter a valid response: n
Invalid response. Please try again.
Please enter a valid response: yes
Thank you for your response.
```

Simple Run2:

```
Please enter a valid response: y
Invalid response. Please try again.
Please enter a valid response: n
Invalid response. Please try again.
Please enter a valid response: no
Thank you for your response.
```

- **Solution:**

```
while True:
    user_input = input("Please enter a valid response: ")
    if user_input == "yes" or user_input == "no" :
      print("Thank you for your response.")
      break
    else:
      print("Invalid response. Please try again.")
      continue
```



for loop

for loop

- In Python, the for loop is designed **to work with a sequence of data items**. When the statement executes, it iterates once for each item in the sequence.
- **Syntax:**

```
for variable in [value1, value2, ...]:
        statements
```

- **The for statement executes in the following manner:**
  - The *variable* is assigned the first value in the list, then the statements that appear in the block are executed.
  - Then, *variable* is assigned the next value in the list, and the statements in the block are executed again.
  - This continues until *variable* has been assigned the last value in the list.



for loop (Example (1))

Program:

```
days = ['Su','Mo','Tu','We','Th','Fr','Sa']
for i in days:
    print(i)
```

The for-loop can be used on any type of iterable structure, such as a list, tuple, str, dict, or file.

Output:

```
Su
Mo
Tu
We
Th
Fr
Sa
```



for loop (Example (2))

Program:

```
for x in [1,2,3,4,5]:
    print(x)
```

Output:

```
1
2
3
4
5
```



for loop

- We can also use for loops for operations other than printing to a screen.

Program:

```
numbers = [1,5,3,7,6,8]
total = 0
for i in numbers:
    total = total + i
print(total)
```

Output:

```
30
```



for loop

- These loops are called "**definite loops**" because they execute an exact number of times.
- Definite loops (for loops) have explicit iteration variables that change each time through a loop. These iteration variables move through the sequence or set.
- We say that "**definite loops iterate through the members of a set**".



Example

Program 1:

```
friends = ["Ahmad", "Ali", "Saad"]
for friend in friends :
    print("Good Morning:", friend)
print('Done!')
```

Output 1:

```
Good Morning: Ahmad
Good Morning: Ali
Good Morning: Saad
Done!
```

Program 2:

```
for i in "great":
        print(i)
```

Output 2:

```
g
r
e
a
t
```



range( ) function

The ***range()*** function generates a list of numbers, which is generally used to iterate over within for loops.

- Syntax: **range(*start, stop, step*)**



Examples

```
for n in range(10):
    print(n)
```

Output:

```
0
1
2
3
4
5
6
7
8
9
```

```
for n in range(1,10):
    print(n)
```

Output:

```
1
2
3
4
5
6
7
8
9
```

```
for n in range(1,10,2):
    print(n)
```

Output:

```
1
3
5
7
9
```



range( ) function

Program 1:

```
for j in range(4):
    print ("# ")
```

Output:

```
#
#
#
#
```

Program 2:

```
for j in range(4):
    print("# ",end="")
```

Output:

```
# # # #
```

To print in horizontal not vertical line we use (end="")



Try out

Write a program that displays the numbers 1 through 10 and their respective squares.

```
print("Number\t\tSquare")
for i in range(1,11):
    print(i,"\t\t", i ** 2)
```

Number	Square
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100



Try out

What will the following codes display?

Q1:
```
for number in range(6):
    print(number)
```

Q2:
```
for number in range(2, 6):
    print(number)
```

Q3:
```
for number in range(0, 501, 100):
    print(number)
```

Q4:
```
for number in range(10, 5, −1):
    print(number)
```



The pass statement

- The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

The pass statement is a null operation; nothing happens when it executes.

- Program:

```
for i in range(0,10):
    pass
print("Good bye!")
```

- Output:

```
Good bye
```



Finding the largest –Maximum value in a list or sequence

Program:

```
largest = None
for itervar in [3, 41, 12, 9, 74, 15]:
    if largest == None or itervar > largest :
        largest = itervar
print('Largest:', largest)
```

- Before the loop, we set largest to the constant **None**.
- **None** is a special constant value which we can store in a variable to mark the variable as "empty"

Output:

Largest: 74



Finding the Smallest - Minimum value in a list or sequence

Program:

```
smallest = None
for itervar in [3, 41, 12, 9, 74, 15]:
    if smallest is None or itervar < smallest:
        smallest = itervar
print('Smallest:', smallest)
```

Output:

Smallest= 3



Nested for loop

- A nested loop is a loop that is inside another loop.

Program:

```
for i in range(4):
    print("#",end=" ")
print()
for j in range(4):
    print("#",end=" ")
print()
for k in range(4):
    print("#",end=" ")
print()
```



```
for i in range(3):
    for j in range(4):
        print("#",end=" ")
    print()
```

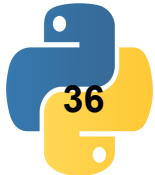
```
# # # #
# # # #
# # # #
```

Nested for loop

- What will the below code produce?

```
for x in range(1,11):
    for y in range(1,11):
        print(y,"*", x , "=",x*y)
```

- To get the total number of iterations of a nested loop, multiply the number of iterations of all the loops.



Any Questions?

Thank you for listening ☺

